CTF Writeups Guide

How to solve challenges, write them up, and publish them safely with `ctf.py`, step by step, without needing Claude.

What's in this guide

#	Section	Use it when
1	How the repo works	You want the big picture in two minutes
2	One-time setup	New computer or fresh clone
3	During a CTF: solving	The event has started
4	Writing the README	You solved something (or gave up)
5	Live CTFs: hiding flags	The event or platform is still running
6	Rebuild, check, commit, push	Every time you finish a writeup
7	When an event ends	Flags can finally be shown
8	Troubleshooting	Something didn't work
9	Cheat sheet and checklist	Quick reference; print this page

1. How the repo works

- **One challenge = one folder** containing a `README.md` (the writeup).
- Folders follow `<event>/<category-or-series>/<challenge>/`, e.g. `cylabCTF/web/old-sessions/`. Lowercase, dashes instead of spaces.
- The top of every writeup has an **info block** between two `---` lines (title, difficulty, flag, tags...). **That block is the source of truth.**
- `ctf.py` reads every info block and **builds all index tables for you**: the root README, each event README and each category README. You never edit a table by hand.
- An event folder that contains a `LIVE.md` file is a **live CTF**. Its flags are kept off GitHub automatically (Section 5).

```
CTFs/
  README.md            <- list of events (auto-built)
  ctf.py               <- the tool
  _templates/          <- writeup template, category add-ons, scratch-notes template
  .flags.local.json    <- real flags of live CTFs (on YOUR machine only, never pushed)
  cylabCTF/
    LIVE.md            <- marks this event as live: flags hidden
    README.md          <- event index (auto-built)
    web/
      README.md        <- category index (auto-built)
      old-sessions/
        README.md      <- the writeup
        files/         <- challenge files you downloaded (optional)
        notes.md       <- your rough notes (optional)
```

2. One-time setup

Do this once per computer. Open a terminal (PowerShell or Git Bash) and run:

```
git clone https://github.com/mcmnyages/CTF-WriteUps.git CTFs
cd CTFs
python ctf.py install-hook     # blocks any commit that leaks a live-CTF flag
python ctf.py check            # should print: No live-CTF flags found
```

> **Tip:** If `python` isn't found on Windows, use `py` instead (e.g. `py ctf.py check`). Everything in this guide works the same way.

> **Important:** The hook is **not** copied by `git clone`. Every teammate and every new computer needs `python ctf.py install-hook` once. If you skip it, nothing stops a flag from being pushed by mistake.

3. During a CTF: solving

Step A: pull, then create the folder

```
git pull                                          # start from the latest version
python ctf.py new cylabCTF/web/my-challenge       # creates the folder + blank writeup
```

If the event folder has a `LIVE.md`, the tool reminds you that the flag will be hidden. For a brand-new event, just use a new first folder name (e.g. `2027-some-ctf/pwn/baby-rop`). If it's a live event, mark it live straight away (Section 5).

Step B: save everything while the server is up

- Put downloaded challenge files in a `files/` folder inside the challenge folder. Servers and downloads usually disappear when the event ends.
- Keep messy notes in `notes.md` next to the writeup. Copy `_templates/notes-scratch.md` and paste commands and output with a timestamp as you go. Don't format anything yet.
- Take screenshots of web pages, OSINT pivots and anything else that might change.

Step C: a solving routine that works for most challenges

1. **Read the description twice.** The title and wording are usually hints (e.g. "Old Sessions" pointed at session expiry).
2. **Recon first.** Web: browse every page, view source, check cookies, `robots.txt`, comments and hidden endpoints. Files: `file`, `strings`, `exiftool`, `binwalk`. Binaries: `checksec`, then Ghidra.
3. **Name the vulnerability class** before attacking (IDOR, SQLi, SSTI, weak crypto...). `_templates/category-sections.md` lists what to look for in each category.
4. **Search your old writeups** for the technique: `grep -rl "jwt" --include=README.md .`
5. **Log dead ends in notes.md.** They become the most useful section for teammates later.
6. **Time-box.** If you're stuck for about 45 minutes, switch challenges or ask a teammate, then come back.

4. Writing the README

Open the `README.md` that `ctf.py new` created. It already has the right shape. Fill in two parts: the **info block** at the top and the **body**.

4a. The info block (feeds the index tables)

```
---
title: "My Challenge"
event: "cylabCTF (CyLab Academy)"
series: "Standalone"            # or the series name for chained challenges
category: web                   # web | pwn | rev | crypto | forensics | osint | misc ...
difficulty: easy                # easy | medium | hard | insane
points: 100
status: solved                  # solved | unsolved
solved_with: solo               # solo | team | with-AI
author: "your-handle"
date: 2026-11-03
flag: "academy{the_real_flag}"  # write the REAL flag; ctf.py hides it if the event is live
tags: [idor, broken-access-control]
live: auto                      # auto = follow the event | true = always hide | false = always show
---
```

Field	What to know
`title`	Shown in the tables. If it still says `<Challenge Name>`, the table shows exactly that, so replace it.
`status` / `flag`	A challenge counts as solved only if status is `solved` **and** the flag is real. Gave up? Set `status: unsolved` and still write up what you tried.
`tags`	The first 4 appear in the tables. Use the technique names from the category add-ons so grep finds them later.
`live`	Leave as `auto`. Use `true` to hide one flag in a finished event, or `false` to show one in a live event (e.g. a retired challenge).
Keep the `---` lines	The block must be the very first thing in the file. If the tables show the folder path instead of your title, the block is broken or missing.

4b. The body: write for two readers

A **beginner** should be able to follow along, and an **expert** should be able to read the TL;DR and stop. The minimum useful writeup is **TL;DR + Key Insight + Solution + Flag**.

Section	What goes in it
TL;DR	One or two sentences: `exposed /sessions -> reuse admin cookie -> flag`.
Background	2 to 4 plain-language bullets on the concept ("What is a session?").
The Challenge	The original description and the URL or nc address. Mention the files in `files/`.
Key Insight	The single "aha" moment and the clue that led to it. Future-you forgets this first.
Solution	Numbered steps. For each one: what you did, **why**, and what came out. Paste real commands, requests and output.

Section	What goes in it
Why It Worked	The underlying weakness in one paragraph, plus a one-line fix.
Flag	The flag in a code block. Write the real one; it's hidden automatically for live events.
Takeaways	What you learned and what would make you faster next time.
Optional	Recon, Dead Ends, Solve Script (save it as `solve.py`), Chain Note. Delete what you don't use.

> **Tip:** Using AI to help solve? That's fine. Set `solved_with: with-AI`, but make sure you can explain every step yourself. The writeup is how you learn it.

5. Live CTFs: hiding flags

A CTF is **live** if its challenges can still be solved for points or for learning: an ongoing competition, or a permanent practice platform such as CyLab Academy. Publishing those flags spoils the challenge for others and usually breaks the platform's rules. The writeup (the method) can stay; the flag must not reach GitHub.

```
python ctf.py live cylabCTF      # mark a whole event as live (creates cylabCTF/LIVE.md)
python ctf.py                    # hides flags in live events + rebuilds every table
python ctf.py check              # double-check: exits with an error if a live flag is visible
```

What happens when you run `python ctf.py` on a live event:

1. The real flag from the info block is copied into `.flags.local.json` (git-ignored, only on your machine).
2. **Every** copy of that exact flag in the writeup, in the info block and the Flag section alike, is replaced with `[FLAG HIDDEN]`.
3. The index tables show **hidden (live CTF)** instead of the flag.

> **Important:** The tool only hides the exact flag string. **Other give-aways are your job:** admin session cookies, tokens, passwords, or a flag split into pieces. Replace them by hand with something like `[SESSION ID HIDDEN — live CTF]`. The pre-commit hook also flags anything that *looks* like `prefix{...}` in a live writeup.

> **Important:** Back up `.flags.local.json` (e.g. a private note or password manager). It's the only copy of your live flags. If you lose it, `reveal` can't restore them.

6. Rebuild, check, commit, push

Do this every time you finish (or update) a writeup:

```
python ctf.py                    # hide live flags + rebuild all index tables
python ctf.py check              # must say: No live-CTF flags found
git status                       # look at what changed; .flags.local.json must NOT be listed
git add -A
git commit -m "writeup: my-challenge (web)"
git pull --rebase                # pick up teammates' work
git push
```

If the commit is **refused** with "Possible live-CTF flags found", the hook did its job. Run `python ctf.py`, fix anything it lists by hand, then commit again. Never bypass it with `--no-verify`.

> **Important:** If a flag **was** pushed by mistake: deleting it in a new commit is not enough, because it's still in the history. If it was your latest commit, fix the file, then run `git commit --amend` and `git push --force-with-lease`. If it's older, ask for help: the history must be rewritten. Assume it has been seen.

7. When an event ends

Once a competition is over (and its rules allow writeups with flags), put the real flags back:

```
python ctf.py reveal my-event    # restore flags, delete LIVE.md, rebuild tables
git add -A && git commit -m "my-event ended: reveal flags" && git push
```

If `reveal` says *No saved flag for …*, that flag was hidden on another computer. Copy the real flag in by hand where `[FLAG HIDDEN]` appears. Practice platforms like CyLab Academy normally stay live, so don't reveal those.

8. Troubleshooting

Problem	Fix
Table row shows a folder path or `<Challenge Name>`	The info block is broken or still a placeholder. It must start on the first line with `---` and end with `---`. Fill in `title`, then run `python ctf.py`.
Solved challenge shows as unsolved	Check `status: solved` and that `flag` is not `FLAG{...}` or empty.
Commit refused: live-CTF flags found	Run `python ctf.py`. Edit any remaining lines it prints (file:line). Commit again.
`python` not recognized	Use `py` or `python3`, or install Python from python.org.
Warnings: *LF will be replaced by CRLF*	Harmless on Windows. Ignore them.
`git push` rejected (non-fast-forward)	A teammate pushed first: `git pull --rebase`, then `git push`.
Merge conflict inside an index table	Accept either side, then run `python ctf.py`. It regenerates the table correctly.
New category has an ugly heading	Add it to `FOLDER_TITLES` near the top of `ctf.py`.

9. Cheat sheet

Command	What it does
`python ctf.py new <event>/<cat>/<name>`	Create a writeup folder from the template
`python ctf.py`	Hide live flags + rebuild every index table (safe to run any time)
`python ctf.py check`	Scan for visible live-CTF flags (error if any)
`python ctf.py install-hook`	Run `check` automatically before each commit (once per clone)
`python ctf.py live <event>`	Mark an event live: hide its flags
`python ctf.py reveal <event>`	Event over: restore flags, un-mark live
`grep -rl "ssti" --include=README.md .`	Find old writeups that used a technique

Before you push: checklist

	Check
[ ]	Info block filled in: title, category, difficulty, status, flag, tags, author, date
[ ]	TL;DR, Key Insight, Solution and Flag sections written; placeholder text deleted
[ ]	Real commands and output pasted; solve script saved as `solve.py` if there is one
[ ]	Live event? No cookies, tokens, passwords or flag pieces left in the text
[ ]	Ran `python ctf.py` and `python ctf.py check` (No live-CTF flags found)
[ ]	`git status` does not list `.flags.local.json`
[ ]	`git pull --rebase`, then `git push`

Stuck with the tool itself? Run `python ctf.py help` to print the built-in usage text.